

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	5
1.1 Описание входных данных.....	6
1.2 Описание выходных данных.....	6
2 МЕТОД РЕШЕНИЯ.....	7
3 ОПИСАНИЕ АЛГОРИТМОВ.....	9
3.1 Алгоритм конструктора класса Character.....	9
3.2 Алгоритм функции displayCharacter.....	9
3.3 Алгоритм метода heal класса Healer.....	10
3.4 Алгоритм функции main.....	10
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	12
5 КОД ПРОГРАММЫ.....	14
5.1 Файл Character.cpp.....	14
5.2 Файл Character.h.....	14
5.3 Файл Healer.cpp.....	15
5.4 Файл Healer.h.....	15
5.5 Файл main.cpp.....	15
6 ТЕСТИРОВАНИЕ.....	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	18

1 ПОСТАНОВКА ЗАДАЧИ

Разработать программную систему на языке C++, моделирующую объект персонажа игры, обладающего именем и внутренними характеристиками. Доступ к данным объекта должен осуществляться с использованием механизма `friend`.

Требования к классу Персонажа

Класс Персонаж должен содержать:

- три свойства:
 - имя героя (тип строковый);
 - уровень здоровья (целочисленный тип);
 - уровень маны (целочисленный тип);
- все свойства должны иметь **закрытый модификатор доступа**;
- конструктор, инициализирующий значения всех свойств.

Использование публичных методов для получения или изменения значений свойств (`get`, `set`) **не допускается**.

Реализовать функцию вывода исходного состояния объекта, которая:

- принимает объект класса Персонаж по константной ссылке;
- выводит на экран имя героя, значение здоровья и значение маны;
- имеет доступ к закрытым членам класса Персонаж.

Реализовать класс Лекарь, который:

- содержит метод, изменяющий состояние объекта класса Персонаж:
 - увеличивает значение здоровья на 20;
 - увеличивает значение маны на 10;
- имеет доступ к закрытым членам класса Персонаж.

Алгоритм работы программы

1. Ввести с клавиатуры имя героя, начальное значение здоровья и начальное значение маны.

2. Создать объект класса Персонаж, используя введённые значения.
3. Вызвать функцию для вывода исходного состояния объекта.
4. Создать объект класса Лекарь.
5. Передать объект класса Персонаж в метод класса Лекарь для изменения его состояния.
6. Повторно вызвать функцию для вывода исходного состояния объекта.

Ограничения

- Прямой доступ к закрытым свойствам класса Персонаж вне дружественных сущностей запрещён.
- Использование публичных геттеров и сеттеров запрещено.
- Обращение к закрытым членам класса Персонаж из функции `main` запрещено.
- Использование механизма `friend` допускается только в рамках требований задания.

1.1 Описание входных данных

Значения через пробел: <Имя> <здоровье> <мана>

1.2 Описание выходных данных

Первая строка: "Hero: <Имя> | Health = <значение здоровья> | Mana = <значение маны>"

Вторая строка: "Hero: <Имя> | Health = <значение здоровья> | Mana = <значение маны>"

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект hero класса Character предназначен для хранения данных игрового персонажа: имени, уровня здоровья и уровня маны с закрытым доступом к свойствам;
- объект healer класса Healer предназначен для выполнения операции лечения над объектом Character через дружественный метод, имеющий доступ к закрытым членам;
- функция main для организации работы программы: ввода данных, создания объектов, вызова функции вывода и метода лечения, демонстрации изменении состояния персонажа;
- функция displayCharacter для вывода исходного и изменённого состояния объекта с доступом к закрытым свойствам;
- ключевое слово friend для предоставления доступа к закрытым членам класса внешним сущностям;
- предварительное объявление для обращения к классу до его описания.

Класс Character:

- свойства/поля:
 - поле хранение имени персонажа:
 - наименование — name;
 - тип — строка;
 - модификатор доступа — private;
 - поле хранение уровня здоровья персонажа:
 - наименование — health;
 - тип — цел;

- модификатор доступа — private;
- о поле хранение уровня маны персонажа:
 - наименование — mana;
 - тип — цел;
 - модификатор доступа — private;
- функционал:
 - о метод Character — конструктор для инициализации всех закрытых свойств объекта переданными значениями.

Класс Healer:

- функционал:
 - о метод heal — метод для изменения состояния объекта Character: увеличивает здоровье на 20 и ману на 10.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	Character			Класс игрового персонажа, содержащий закрытые свойства (имя, здоровье, мана) и предоставляющий доступ к ним только дружественным сущностям	
2	Healer			Класс лекаря, содержащий метод лечения персонажа. Имеет дружественный доступ к закрытым членам класса Character для изменения его состояния	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса `Character`

Функционал: конструктор для инициализации всех закрытых свойств объекта переданными значениями.

Параметры: строка `n` - имя персонажа; цел `h` - начальное значение здоровья; цел `m` - начальное значение маны.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса `Character`

№	Предикат	Действия	№ перехода
1		иниц. поля <code>name</code> значением парам. <code>n</code>	2
2		иниц. поля <code>health</code> значением парам. <code>h</code>	3
3		иниц. поля <code>mana</code> значением парам. <code>m</code>	Ø

3.2 Алгоритм функции `displayCharacter`

Функционал: дружественная функция для вывода состояния персонажа, имеет доступ к закрытым членам.

Параметры: конст. ссылка `c` на объект `Character`.

Возвращаемое значение: функция не возвращает значения.

Алгоритм функции представлен в таблице 3.

Таблица 3 – Алгоритм функции *displayCharacter*

№	Предикат	Действия	№ перехода
1		вывод на консоль "Hero: ", c.name, " Health = ", c.health, " Mana = ", c.mana	Ø

3.3 Алгоритм метода *heal* класса *Healer*

Функционал: метод для изменения состояния объекта *Character*: увеличивает здоровье на 20 и ману на 10.

Параметры: нет.

Возвращаемое значение: ссылка с на объект *Character*.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода *heal* класса *Healer*

№	Предикат	Действия	№ перехода
1		прибавить к значению поля c.health 20	2
2		прибавить к значению поля c.mana 10	Ø

3.4 Алгоритм функции *main*

Функционал: организация работы программы: ввод данных, создание объектов, вызов функции вывода и метода лечения, демонстрация изменении состояния персонажа.

Параметры: нет.

Возвращаемое значение: *int* - код возврата программы.

Алгоритм функции представлен в таблице 5.

Таблица 5 – Алгоритм функции *main*

№	Предикат	Действия	№ перехода
1		объявл. строковой перемен. name	2
2		объявл. цел. перемен. health и mana	3
3		ввод значений name, health, mana с клавиатуры	4
4		создание объекта hero класса Character с параметрами name, health, mana	5
5		вызов функции displayCharacter с передачей объекта hero	6
6		созд. объекта healer класса Healer	7
7		вызов метода heal объекта healer с передачей объекта hero	8
8		вызов функции displayCharacter с передачей объекта hero	9
9		возврат 0	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-2.

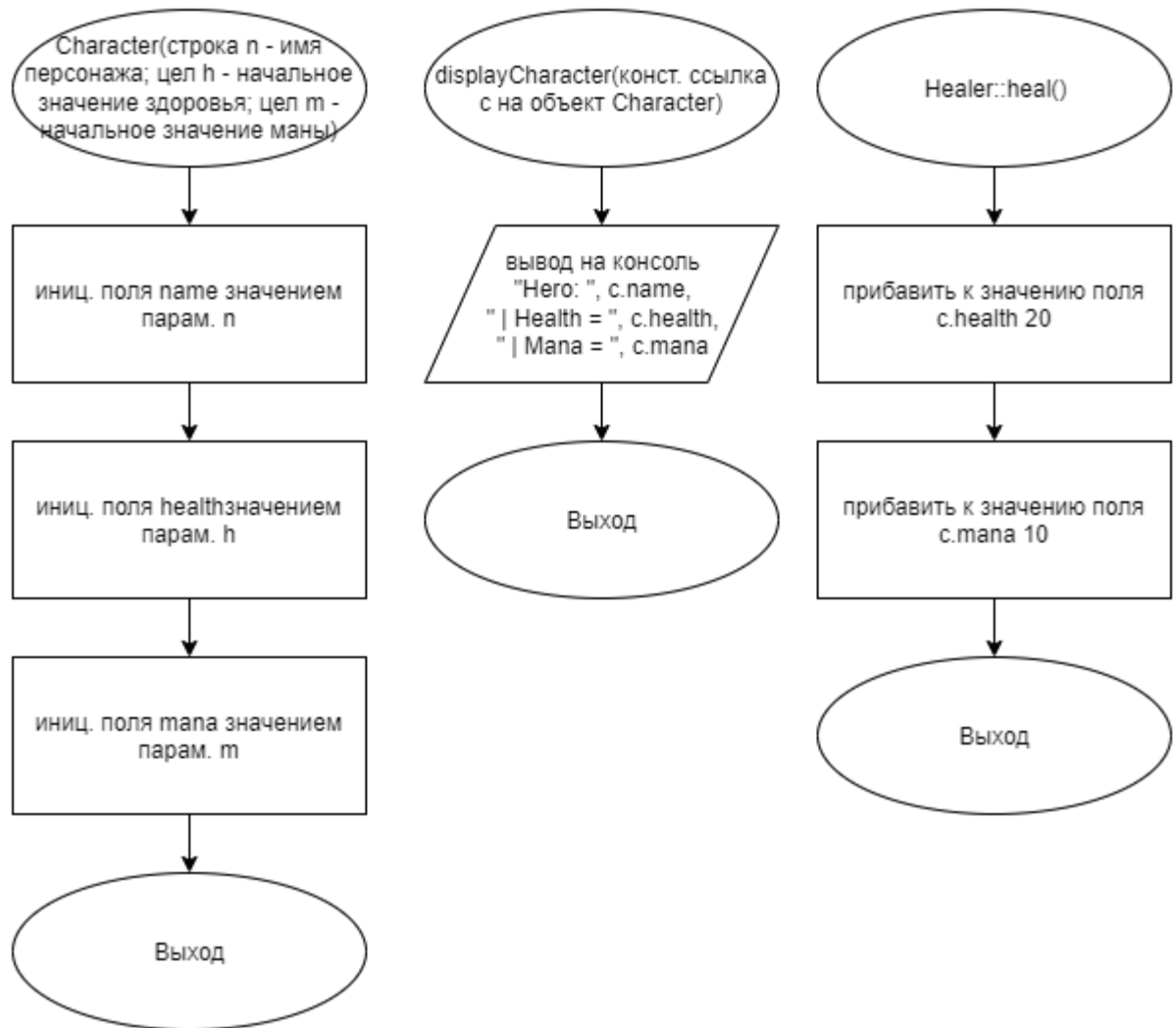


Рисунок 1 – Блок-схема алгоритма

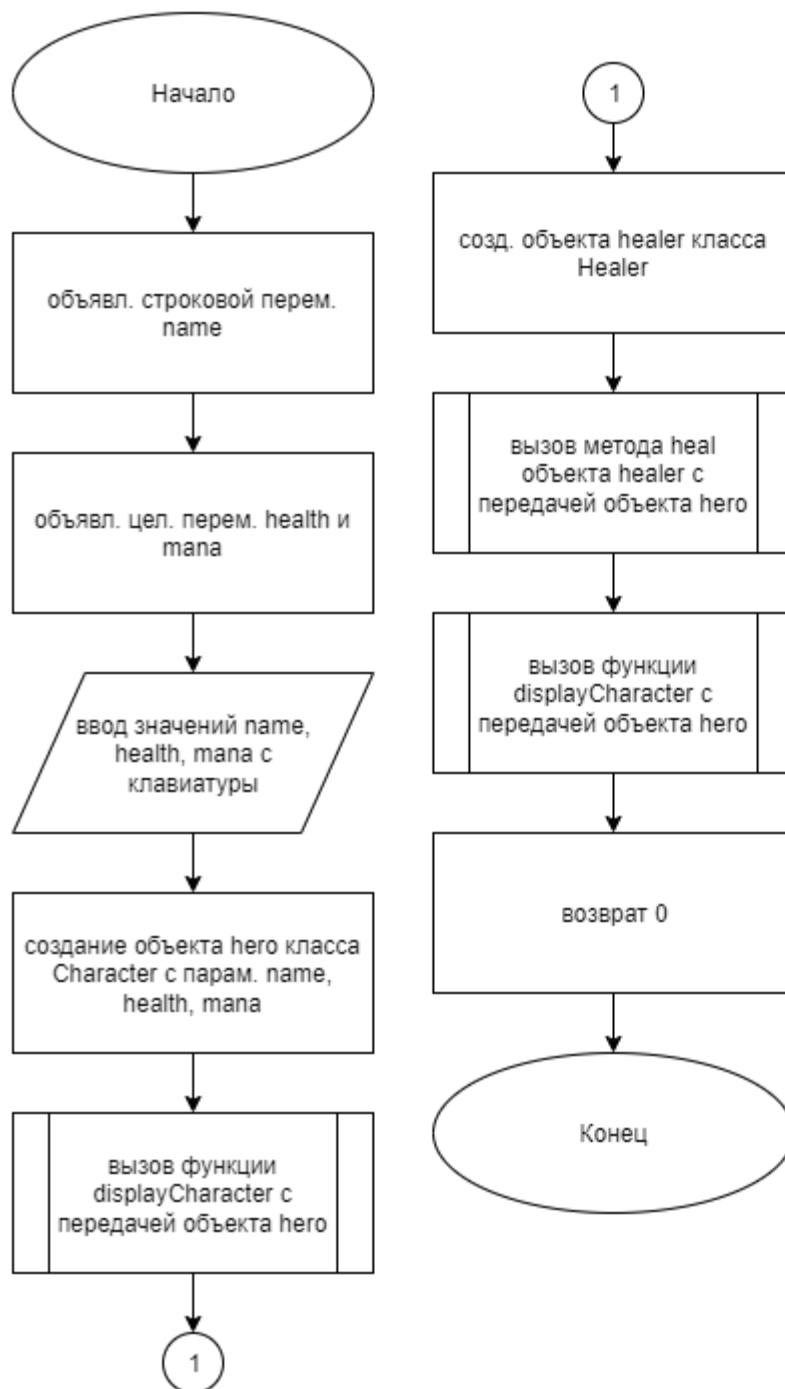


Рисунок 2 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Character.cpp

Листинг 1 – Character.cpp

```
#include "Character.h"
#include <iostream>

Character::Character(std::string n, int h, int m)
    : name(n), health(h), mana(m) {}

void displayCharacter(const Character& c) {
    std::cout << "Hero: " << c.name << " | Health = " << c.health << " | Mana
= " << c.mana << std::endl;
}
```

5.2 Файл Character.h

Листинг 2 – Character.h

```
#ifndef __CHARACTER__H
#define __CHARACTER__H
#include <string>

class Healer;

class Character {
    std::string name;
    int health;
    int mana;
public:
    Character(std::string n, int h, int m);
    friend void displayCharacter(const Character& c);
    friend class Healer;
};

#endif
```

5.3 Файл Healer.cpp

Листинг 3 – Healer.cpp

```
#include "Healer.h"
#include "Character.h"

void Healer::heal(Character& c) {
    c.health += 20;
    c.mana += 10;
}
```

5.4 Файл Healer.h

Листинг 4 – Healer.h

```
#ifndef __HEALER__H
#define __HEALER__H

class Character;

class Healer {
public:
    void heal(Character& c);
};

#endif
```

5.5 Файл main.cpp

Листинг 5 – main.cpp

```
#include <stdlib.h>
#include <stdio.h>
#include <iostream>
#include <string>
#include "Character.h"
#include "Healer.h"

int main()
{
    std::string name;
    int health, mana;
```

```
    std::cin >> name >> health >> mana;  
    Character hero(name, health, mana);  
    displayCharacter(hero);  
  
    Healer healer;  
    healer.heal(hero);  
    displayCharacter(hero);  
  
    return 0;  
}
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 6.

Таблица 6 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Andrew 80 11	Hero: Andrew Health = 80 Mana = 11 Hero: Andrew Health = 100 Mana = 21	Hero: Andrew Health = 80 Mana = 11 Hero: Andrew Health = 100 Mana = 21
John 0 0	Hero: John Health = 0 Mana = 0 Hero: John Health = 20 Mana = 10	Hero: John Health = 0 Mana = 0 Hero: John Health = 20 Mana = 10

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoe_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).